

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

Members:

YADAV Anshuman Krishna (exercise 1)

MAHALINGAM Nithees (exercise 2)

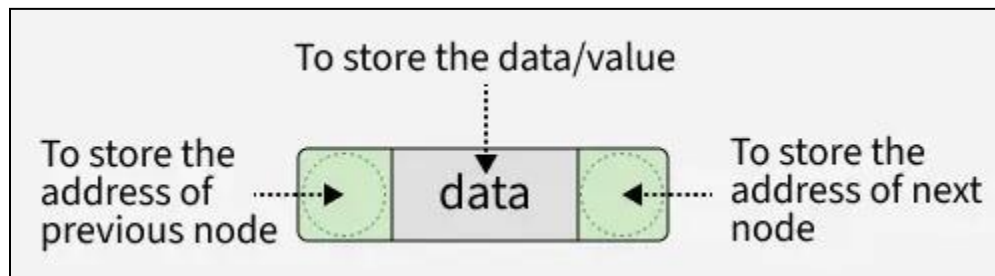
SARAVANAN Arun Prasath (exercise 3)

Github Link:

https://github.com/anshuman-krishna/advanced-algorithms-and-programming/tree/LAB_03_LinearDS

Exercise 1: YADAV Anshuman Krishna

Content Feed Navigation with Doubly Linked List



1. StoryNode

```
STRUCTURE StoryNode
story_id : String
user_id : String
content_preview : String
timestamp : Integer
views : Integer
next : StoryNode
prev : StoryNode
END STRUCTURE
```

2. DoublyLinkedList

```
STRUCTURE DoublyLinkedList
head : StoryNode
tail : StoryNode
current : StoryNode
size : Integer
END STRUCTURE
```

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

Time Complexity : $O(1)$

Space Complexity : $O(1)$

3. Core

A. add_story

```
PROCEDURE add_story(list, node)
```

```
node.next <- NULL
```

```
node.views <- 0
```

```
IF
```

```
list.head = NULL THEN
```

```
node.prev <- NULL
```

```
list.head <- node
```

```
list.tail <- node
```

```
list.current <- node
```

```
ELSE
```

```
node.prev <- list.tail
```

```
list.tail.next <- node
```

```
list.tail <- node
```

```
END IF
```

```
list.size <- list.size + 1
```

```
END PROCEDURE
```

Time Complexity : $O(1)$

Space Complexity : $O(1)$

B. remove_story

```
PROCEDURE remove_story(list, story_id)
```

```
temp <- list.head
```

```
WHILE temp != NULL DO
```

```
IF temp.story_id = story_id THEN
```

```
IF temp.prev != NULL THEN
```

```
temp.prev.next <- temp.next
```

```
ELSE
```

```
list.head <- temp.next
```

```
END IF
```

```
IF temp.next != NULL THEN
```

```
temp.next.prev <- temp.prev
```

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

```
ELSE
list.tail <- temp.prev
END IF

IF list.current = temp THEN
list.current <- temp.next
END IF

list.size <- list.size - 1
Break
END IF
temp <- temp.next
END WHILE
END PROCEDURE
```

Time Complexity : $O(n)$

Space Complexity : $O(1)$

C. move_forward

```
FUNCTION move_forward(list) -> String
IF list.current != NULL AND list.current.next != NULL THEN
list.current <- list.current.next
RETURN list.current.content_preview
END IF
RETURN "End of feed"
```

Time Complexity : $O(1)$

Space Complexity : $O(1)$

D. move_backward

```
FUNCTION move_backward(list) -> String
IF list.current != NULL AND list.current.prev != NULL THEN
list.current <- list.current.prev
RETURN list.current.content_preview
END IF
RETURN "Beginning of feed"
```

Time Complexity : $O(1)$

Space Complexity : $O(1)$

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

E. **jump_to(story_id):**

```
PROCEDURE jump_to(list, story_id)
temp <- list.head
WHILE temp != NULL DO
IF temp.story_id = story_id THEN
list.current <- temp
Break
END IF
temp <- temp.next
END WHILE
END PROCEDURE
```

Time Complexity : $O(n)$

Space Complexity : $O(1)$

F. **insert_after(current_id, new_story)**

```
PROCEDURE insert_after(list, current_id, new_story)
temp <- list.head
WHILE temp != NULL DO
IF temp.story_id = current_id THEN
new_story.next <- temp.next
new_story.prev <- temp

IF temp.next != NULL THEN
temp.next.prev <- new_story
ELSE
list.tail <- new_story
END IF

temp.next <- new_story
list.size <- list.size + 1
Break
END IF
temp <- temp.next
END WHILE
END PROCEDURE
```

Time Complexity : $O(n)$

Space Complexity : $O(1)$

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

G. display_around_current(k)

FUNCTION display_around_current(list, k) -> Array

results <- empty list

temp <- list.current

count <- 0

WHILE temp != NULL AND count < k DO

temp <- temp.prev

count <- count + 1

END WHILE

IF temp = NULL THEN

temp <- list.head

END IF

total_to_show <- (k * 2) + 1

count <- 0

WHILE temp != NULL AND count < total_to_show DO

results.Add(temp.content_preview)

temp <- temp.next

count <- count + 1

END WHILE

RETURN results

Time Complexity : $O(k)$

Space Complexity : $O(k)$

4. Engagement Tracking:

A. track_view(current)

PROCEDURE track_view(list)

IF list.current != NULL THEN

list.current.views <- list.current.views + 1

END IF

END PROCEDURE

Time Complexity : $O(1)$

Space Complexity : $O(1)$

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

B. most_viewed

```
FUNCTION most_viewed(list) -> StoryNode
IF list.head = NULL THEN
RETURN NULL
END IF
```

```
max_node <- list.head
temp <- list.head.next
```

```
WHILE temp != NULL DO
IF temp.views > max_node.views THEN
max_node <- temp
END IF
temp <- temp.next
END WHILE
```

```
RETURN max_node
```

Time Complexity : $O(n)$

Space Complexity : $O(1)$

C. Reorder_by_views

```
PROCEDURE reorder_by_views(list)
IF list.head = NULL OR list.head.next = NULL THEN
RETURN
END IF
```

```
swapped <- TRUE
WHILE swapped = TRUE DO
swapped <- FALSE
current_node <- list.head
```

```
WHILE current_node.next != NULL DO
next_node <- current_node.next
IF current_node.views < next_node.views THEN
temp_views <- current_node.views
current_node.views <- next_node.views
next_node.views <- temp_views
```

```
temp_id <- current_node.story_id
current_node.story_id <- next_node.story_id
```

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

```
next_node.story_id <- temp_id

temp_preview <- current_node.content_preview
current_node.content_preview <- next_node.content_preview
next_node.content_preview <- temp_preview

swapped <- TRUE
END IF
current_node <- current_node.next
END WHILE
END WHILE
END PROCEDURE
```

Time Complexity : $O(n^2)$

Space Complexity : $O(1)$

Complexity Analysis Questions:

1. Compare time complexity of forward/backward navigation in singly vs doubly linked list?

Answer:

Singly Linked List: Moving forward is $O(1)$.

Moving backward is $O(n)$ because a singly linked list only has forward pointers.

To move back one step, we have to start at the head and loop all the way through the list to find the node that points to your current node.

Doubly Linked List: Both moving forward and backward are $O(1)$.

We just access the next or prev pointers instantly.

2. What is the time complexity of removing a node given only a pointer to that node?

Answer:

In a **doubly linked list**, it is $O(1)$. Because we have a pointer to the target node, we instantly have access to its previous and next neighbours. Hence we can just bypass the target node by linking its neighbours directly to each other.

In a **singly linked list**, it would be $O(n)$ because we don't know which node points to the target node without scanning from the beginning.

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

3. How much extra memory per node does a doubly linked list require compared to singly linked?

Answer:

It requires one extra pointer variable per node (the previous pointer). Primarily depending on our system architecture, this means an overhead of 4 bytes (on 32-bit systems) or 8 bytes (on 64-bit systems) of extra memory per node.

4. In what scenarios would the bidirectional navigation justify the memory overhead?

Answer:

It is justified in any application where frequent back-and-forth movement or constant tail-end modifications are required. Good examples include browser history back/forward buttons, undo/redo mechanisms in text editors, navigating music playlists, or a social media "Story Highlights" feed where users rapidly tap backwards to rewatch recent content.

Code Related Queries

Test Cases

Input	Expected Output	Reason
Normal Addition & Forward Nav: - Add Story 1, Add Story 2. - Call <code>move_forward()</code>	Output: "Story 2 Preview"	Tests if the tail pointer updates correctly and if the next linkage works seamlessly when navigating.

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

Edge Case (Backward Nav at Head): - List has 3 stories. Current is at Story 1. - Call <code>move_backward()</code>	Output: "Beginning of feed"	Ensures the algorithm safely handles boundary limits without throwing a Null Pointer Exception.
Insertion in the Middle: - Add Story 1, 3. - Call <code>insert_after("1", Story 2)</code>	Feed order becomes 1 -> 2 -> 3	Verifies that inserting a node correctly updates both the prev and nextpointers of the surrounding nodes without breaking the chain.
Edge Case (Remove Current Node): - Current is Story 2. - Call <code>remove_story("2")</code>	Current pointer shifts to Story 3	Tests standard deletion but also ensures the active current pointer safely falls back to the next available node instead of pointing to a deleted object.
Sorting by Views: - Story A (5 views), Story B (10 views), Story C (2 views). - Call <code>reorder_by_views()</code>	Feed order: B -> A -> C	Verifies the bubble sort logic correctly swaps the data inside the nodes based on the view count integer.

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

Advanced Complexity Analysis

- **Addition (add_story):** Time $O(1)$, Space $O(1)$.
Because we explicitly track the tail pointer, we never have to traverse the list to add a new story. We simply attach it to the end instantly.
 - **Navigation (move_forward / move_backward):** Time $O(1)$, Space $O(1)$.
The primary advantage of the Doubly Linked List is bidirectional $O(1)$ navigation. We just follow the immediate pointer.
 - **Searching / Inserting / Removing by ID:** Time $O(n)$, Space $O(1)$.
Unless we already have the memory address of the specific node, we are forced to linearly scan the list from the head to find the target ID.
 - **Contextual Display (display_around_current):** Time $O(k)$, Space $O(k)$.
The time scales linearly with k (the radius of stories we want to fetch).
 - **Sorting (reorder_by_views):** Time $O(n^2)$, Space $O(1)$.
We implemented an in-place Bubble Sort. While it uses no extra memory, repeatedly scanning the list to swap adjacent nodes becomes highly inefficient as the number of stories grows.
-

Reflection

Implementing this feed using a Doubly Linked List made the trade-offs of the data structure incredibly obvious to us. The advantage here is how effortlessly we can move backwards and forwards; something that would be a massive headache (and require $O(n)$ time) in a Singly Linked List is handled instantly here.

However, we also realized the cost. Managing two pointers means there is a much higher chance of creating a bug when inserting or deleting nodes, as we have to carefully stitch four different connections back together. If the app needed to constantly re-rank millions of stories by popularity in real-time, we would likely need to pivot to a different structure, like a Max Heap or a Tree.

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

Exercise 2: MAHALINGAM Nithees

Activity Feed Processing with Stacks and Queues

Part A : Recent Activity Stack (LIFO)

Structure:

```
Structure Node
activity
next
End Structure
```

```
Structure ActivityStack
top
size
End Structure
```

Initialize Stack:

```
Function createStack()
stack.top <- NULL
stack.size <- 0
return stack
End
```

Push:

```
Function push(stack, activity)
newNode <- new Node
newNode.activity <- activity
newNode.next <- stack.top
```

```
stack.top <- newNode
stack.size <- stack.size + 1
End
```

Pop:

```
Function pop(stack)
If stack.top = NULL
return "Stack Empty"
EndIf
```

```
temp <- stack.top
```

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

```
stack.top <- stack.top.next  
stack.size <- stack.size - 1
```

```
return temp.activity  
End
```

Peek:

```
Function peek(stack)  
If stack.top = NULL  
return "Stack Empty"  
EndIf
```

```
return stack.top.activity  
End
```

Is_empty:

```
Function is_empty(stack)  
If stack.top = NULL  
return TRUE  
Else  
return FALSE  
End
```

Size:

```
Function size(stack)  
return stack.size  
End
```

Display_recent:

```
Function display_recent(stack)  
current <- stack.top
```

```
While current != NULL  
print current.activity  
current <- current.next  
End
```

Undo operation:

```
Function undo(stack, undoStack)  
activity <- pop(stack)
```

```
If activity != "Stack Empty"
```

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

```
push(undoStack, activity)
EndIf
End
```

Redo Operation:

```
Function redo(stack, undoStack)
activity <- pop(undoStack)
```

```
If activity != "Stack Empty"
push(stack, activity)
EndIf
End
```

Part B : Notification Queue (FIFO)

Structure:

```
Structure QNode
notification
next
End Structure
```

```
Structure NotificationQueue
front
rear
size
End Structure
```

Enqueue:

```
Function enqueue(queue, notification)
```

```
newNode <- new QNode
newNode.notification <- notification
newNode.next <- NULL
```

```
If queue.rear = NULL
queue.front <- newNode
queue.rear <- newNode
Else
queue.rear.next <- newNode
queue.rear <- newNode
EndIf
```

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

```
queue.size <- queue.size + 1  
End
```

Dequeue:

```
Function dequeue(queue)
```

```
If queue.front = NULL  
return "Queue Empty"  
EndIf
```

```
temp <- queue.front  
queue.front <- queue.front.next
```

```
If queue.front = NULL  
queue.rear <- NULL  
EndIf
```

```
queue.size <- queue.size - 1  
return temp.notification  
End
```

Front:

```
Function front(queue)
```

```
If queue.front = NULL  
return "Queue Empty"  
EndIf
```

```
return queue.front.notification  
End
```

Is_empty:

```
Function is_empty(queue)
```

```
If queue.front = NULL  
return TRUE  
Else  
return FALSE  
End
```

Size:

```
Function size(queue)
```

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

```
return queue.size
```

```
End
```

Display_pending:

```
Function display_pending(queue)
```

```
current <- queue.front
```

```
While current != NULL
```

```
print current.notification
```

```
current <- current.next
```

```
End
```

Priority Enqueue:

```
Function priority_enqueue(queue, notification)
```

```
newNode <- new QNode
```

```
newNode.notification <- notification
```

```
newNode.next <- queue.front
```

```
queue.front <- newNode
```

```
If queue.rear = NULL
```

```
queue.rear <- newNode
```

```
EndIf
```

```
queue.size <- queue.size + 1
```

```
End
```

Part C : Integrated Feed Processing System

Structure:

```
Structure FeedProcessor
```

```
recent_stack
```

```
notification_queue
```

```
processed_log
```

```
End Structure
```

Process_notifications:

```
Function process_notifications(system)
```

```
notification <- dequeue(system.notification_queue)
```

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

```
If notification != "Queue Empty"
push(system.recent_stack, notification)
EndIf
End
```

Batch_process:

```
Function batch_process(system, k)
For i <- 1 to k
notification <- dequeue(system.notification_queue)
If notification = "Queue Empty"
break
EndIf

push(system.recent_stack, notification)
EndFor
End
```

Clear_history:

```
Function clear_history(system)
While NOT is_empty(system.recent_stack)
activity <- pop(system.recent_stack)
enqueue(system.processed_log, activity)
EndWhile
End
```

Get_stats:

```
Function get_stats(system)
print "Recent Activities:", size(system.recent_stack)
print "Pending Notifications:", size(system.notification_queue)
print "Processed Items:", size(system.processed_log)
End
```

Complexity Analysis Questions:

1. Compare time complexity of all operations.

Answer:

Operation	Complexity
push()	O(1)

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

pop()	$O(1)$
peek()	$O(1)$
enqueue()	$O(1)$
dequeue()	$O(1)$
priority_enqueue()	$O(1)$
display_recent	$O(n)$
display_pending()	$O(n)$
batch_process()	$O(k)$

2. For a queue, which implementation (array circular buffer vs. linked list) is more memory efficient for unknown size?

Answer:

Circular Buffer (Array-based Queue) is more memory efficient than a linked list when:

- Maximum size is known
- No extra pointer memory required
- Better cache performance

Linked lists require extra memory for pointers.

3. What is the amortized time complexity of a sequence of n enqueue/dequeue operations?

Answer:

The amortized time complexity of a sequence of n enqueue and dequeue operations is $O(1)$ per operation.

This is because both enqueue and dequeue take constant time. For n operations, the total time is $O(n)$, so the average (amortized) time per operation is $O(1)$.

4. How would you implement a deque (double-ended queue) to support both stack and queue operations?

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

Answer:

A Deque (Double Ended Queue) allows operations at both ends.

Operations:

- push_front()
- push_back()
- pop_front()
- pop_back()

Implementation:

- Doubly Linked List
- Circular Array

Code Related Queries

1. Detailed Complexity Analysis:

Activity Stack:

Operation	Description	Time Complexity
push(activity)	Add activity to top of stack	O(1)
pop()	Remove most recent activity	O(1)
peek()	View most recent activity	O(1)
is_empty()	Check if stack is empty	O(1)
size()	Return stack size	O(1)
display_recent()	Show all activities	O(n)

Space Complexity: O(n)

where n = number of activities stored in stack

Each activity requires a node with:

- data

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

- pointer to next node

Notification Queue:

Operation	Description	Time Complexity
enqueue(notification)	Add notification to rear	$O(1)$
dequeue()	Remove oldest notification	$O(1)$
front()	View next notification	$O(1)$
is_empty()	Check if queue empty	$O(1)$
size()	Return queue size	$O(1)$
display_pending()	Show all notifications	$O(n)$
priority_enqueue()	Insert urgent notification at front	$O(1)$

Space Complexity: $O(n)$

where n = number of notifications

Feed Processing System:

Operation	Description	Time Complexity
process_notifications()	Move notification to stack	$O(1)$
batch_process(k)	Process k notifications	$O(k)$
clear_history()	Move all stack items to log	$O(n)$
get_stats()	Return structure sizes	$O(1)$

Total System Space Complexity:

$$O(n+m)$$

Where,

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

- n = activities in stack
- m = notifications in queue

2. Reflection:

This system models how social media platforms manage user activity feeds and notifications using fundamental data structures.

A stack is used for recent activities because it follows the **Last-In-First-Out (LIFO)** principle, ensuring the most recent actions appear first. Operations like push, pop, and peek allow quick access to the latest activity.

A queue handles notifications using the **First-In-First-Out (FIFO)** principle, ensuring notifications are processed in the order they arrive.

By integrating these structures in a Feed Processor system, notifications are converted into feed activities after processing, while a processed log stores past actions for tracking.

This implementation demonstrates how basic data structures can be combined to simulate real-world systems such as Instagram, Facebook, or Twitter activity feeds.

Key advantages of this design include:

- Efficient **$O(1)$** operations for most actions
- Clear separation between pending notifications and processed activities
- Ability to support undo/redo functionality
- Scalable processing with batch operations

3. Test Cases:

Input	Expected Output	Description
push("Like")	Stack: Like	Basic stack push
push("Comment")	Stack: Comment -> Like	Multiple pushes
pop()	Removed: Comment	Stack pop
peek()	Like	View top element
pop() on empty stack	"Stack Empty"	Edge case
enqueue("New Follower")	Queue: New Follower	Basic enqueue

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

enqueue("New Like")	Queue: Follower -> Like	Multiple enqueue
dequeue()	Removed: New Follower	FIFO behaviour
dequeue() on empty queue	"Queue Empty"	Edge case
priority_enqueue("Urgent message")	Urgent message at front	Priority handling
batch_process(3)	Process first 3 notifications	Batch Processing
clear_history()	Stack moved to log	History clearing

Exercise 3: SARAVANAN Arun Prasath

Engagement-Based Priority Queue

1. Post Structure

```
STRUCT Post
post_id
user_id
content
timestamp
likes
comments
shares
engagement_score
next
END STRUCT
```

2. Calculate Engagement Score

```
FUNCTION calculate_score(post)
score <- (post.likes * 1) + (post.comments * 2) + (post.shares * 3)
RETURN score
END FUNCTION
```

3. Initialize Priority Queue

```
STRUCT PriorityQueue
```

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

```
head <- NULL
count <- 0
END STRUCT
```

4. Enqueue (Insert Post in Sorted Order)

```
FUNCTION enqueue(queue, new_post)
new_post.engagement_score <- calculate_score(new_post)
IF queue.head = NULL OR
new_post.engagement_score > queue.head.engagement_score THEN
new_post.next <- queue.head
queue.head <- new_post
ELSE
current <- queue.head
WHILE current.next != NULL AND
current.next.engagement_score ≥ new_post.engagement_score DO
current <- current.next
END WHILE
new_post.next <- current.next
current.next <- new_post
END IF
queue.count <- queue.count + 1
END FUNCTION
```

5. Dequeue Max (Remove Highest Priority)

```
FUNCTION dequeue_max(queue)
IF queue.head = NULL THEN
RETURN NULL
END IF
temp <- queue.head
queue.head <- queue.head.next
queue.count <- queue.count - 1
RETURN temp
END FUNCTION
```

6. Peek Max (View Highest Priority)

```
FUNCTION peek_max(queue)
IF queue.head = NULL THEN
RETURN NULL
END IF
RETURN queue.head
END FUNCTION
```

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

7. Check if Queue is Empty

```
FUNCTION is_empty(queue)
IF queue.head = NULL THEN
RETURN TRUE
ELSE
RETURN FALSE
END IF
END FUNCTION
```

8. Get Queue Size

```
FUNCTION size(queue)
RETURN queue.count
END FUNCTION
```

9. Update Post Score

```
FUNCTION update_score(queue, post_id, new_likes, new_comments, new_shares)
current <- queue.head
previous <- NULL
WHILE current != NULL DO
IF current.post_id = post_id THEN
IF previous = NULL THEN
queue.head <- current.next
ELSE
previous.next <- current.next
END IF
queue.count <- queue.count - 1
current.likes <- new_likes
current.comments <- new_comments
current.shares <- new_shares
enqueue(queue, current)
RETURN
END IF
previous <- current
current <- current.next
END WHILE
END FUNCTION
```

10. Get Top K Posts

```
FUNCTION get_top_k(queue, k)
current <- queue.head
i <- 0
WHILE current != NULL AND i < k DO
```

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

```
PRINT current.post_id, current.engagement_score
current <- current.next
i <- i + 1
END WHILE
END FUNCTION
```

11. Decay Older Posts

```
FUNCTION decay_older_than(queue, time)
current <- queue.head
WHILE current != NULL DO
IF current.timestamp < time THEN
current.engagement_score <- current.engagement_score * 0.8
END IF
current <- current.next
END WHILE
END FUNCTION
```

Complexity Analysis Questions:

1. Compare time complexity of enqueue for:

Answer:

- **Sorted linked list implementation:** $O(n)$. To keep the list sorted, you must traverse it from the head to find the correct insertion point. In the worst case (inserting a very low score), you visit every node.
- **Unsorted linked list with find-max each time:** $O(1)$. If the list is unsorted, you can simply "push" the new node to the front of the list. (Note: While enqueue is fast, the corresponding dequeue_max would become $O(n)$ because you'd have to search the whole list for the highest score).

Implementation	Enqueue Time	Reason
Sorted Linked List	$O(n)$	Must traverse the list to insert the post in the correct position

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

Unsorted Linked List	$O(1)$	Post can be inserted directly at the beginning
----------------------	--------	--

2. Worst-case time complexity of update_score? How could you optimize it?

Answer:

The worst-case time complexity of the update_score function is $O(n)$. The algorithm must first search through the linked list to find the post with the given post_id. After locating it, the post is removed, its engagement values are updated, and it is inserted again into the correct position.

This can be optimized by using a hash map that stores references to posts using their post_id. With this approach, the system can find the required post instantly without scanning the entire list. Another improvement would be using a heap-based priority queue to manage posts more efficiently.

3. For a social network with millions of posts, would a sorted linked list be practical?

Answer:

No, using a sorted linked list would not be very practical for a large social network. Since every insertion requires scanning the list to find the correct position, the system would become slow when handling millions of posts. Social media platforms also have constant updates in likes, comments, and shares, which means the scores change frequently. Because of this, a more efficient data structure such as a max heap or priority queue would be better suited for handling large amounts of data.

4. Space complexity: How much extra memory per node?

Answer:

Each node in the queue stores information such as the post ID, user ID, content, timestamp, number of likes, comments, shares, engagement score, and a pointer to the next node. Since these values are fixed for every node, the extra memory required per node is constant. Therefore, the space complexity per node is **$O(1)$** .

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

5. If engagement scores update frequently, what data structure would be more efficient?

Answer:

A Max Heap (Binary Heap) would be more efficient.

Advantages:

- Insert: $O(\log n)$
- Remove max: $O(\log n)$
- Peek max: $O(1)$

This makes it suitable for systems where engagement scores change frequently.

Other possible structures:

- Fibonacci Heap (for faster decrease-key operations)
- Balanced Binary Search Trees (AVL / Red-Black Tree)

Code Related Queries

Reflection

This implementation models a priority queue for social media posts based on engagement metrics such as likes, comments, and shares. Each post receives an engagement score, and posts with higher scores appear at the front of the queue.

Using a sorted linked list ensures that the highest engagement post is always available at the head of the queue, allowing quick access through the **peek_max** and **dequeue_max** operations.

The system also supports dynamic updates. When engagement changes, the **update_score** function removes the post, recalculates the score, and reinserts it in the correct position.

Strengths

- Easy to implement and understand
- Always keeps the highest priority element at the front

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

- Supports dynamic score updates

Weaknesses

- Insert operations are slow for large datasets
- Frequent engagement updates cause repeated list traversals
- Not scalable for millions of posts

Complexity Analysis

Let N be the number of posts in the queue.

Enqueue

- **Time:** $O(N)$
- **Space:** $O(1)$

Deque Max

- **Time:** $O(1)$
- **Space:** $O(1)$

Peek Max

- **Time:** $O(1)$
- **Space:** $O(1)$

Update Score

- **Time:** $O(N)$
- **Space:** $O(1)$

Get Top K Posts

- **Time:** $O(K)$
- **Space:** $O(1)$

Decay Older Posts

- **Time:** $O(N)$
- **Space:** $O(1)$

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

Test Cases

Input	Expected Output	Reason
Post A: likes=10, comments=5, shares=2	Score = $10 + 10 + 6 = 26$	Engagement formula check
Queue: [A(26), B(20), C(15)]	peek_max -> A	Highest engagement
dequeue_max	Removes A	Correct priority removal
update_score(B, likes=20)	B moves to head	Score increases
Empty queue	dequeue_max -> NULL	Edge case handling
k = 2 with posts [50,40,30]	TopK -> [50,40]	Correct top K retrieval

Final Question: Project Proposal & Integration

For our future semester project, we are currently planning to build a **Pinterest-style** platform. We really love the idea of creating a visual trending feed combined with friend activity, and the data structures we built in these exercises map perfectly to how a platform like that actually works behind the scenes. Here is how we plan to bring them all together:

- **Priority Queues for Content Ranking:** The main dashboard on an app like Pinterest does not just show the newest posts first but rather shows what is most relevant. We would use our Priority Queue to rank pins and posts.

II.2415 Advanced Algorithms and Programming

Lab 3: Linear Data Structures: Team T24

Every post would get a calculated score based on saves, clicks, and comments. As new posts get added to the database, the priority queue ensures the highest-quality, most trending content naturally floats to the very top of the feed.

- **Doubly Linked Lists for Feed Navigation:** When we click on a specific image to expand it, we usually want to swipe left or right to see related posts. This is exactly where the Doubly Linked List shines. By loading the currently viewed category into a DLL, we can endlessly tap forward and backward through the content. It makes the media browsing experience feel instant without forcing the app to constantly search an array.
- **Stacks and Queues for Activity Processing:** We need to handle background events without lagging the screen. We would use a Queue (FIFO) for the notification system, like when Arun saves a pin or Nithees follows a board. The queue takes these incoming events and delivers them smoothly one by one. Meanwhile, we would use a Stack (LIFO) for the local session history. If we accidentally hide a pin or delete a save, the stack powers a quick undo button, popping our last action right back into place.

A Quick Note on Our Topic:

Honestly, there is a chance we might change our target application before the project officially starts. We noticed that a lot of other teams are also picking a Pinterest-style app, and we might want to pivot to a different idea so our project stands out a bit more.

However, even if we completely pivot, the logic from these three exercises will help us nonetheless. Whether we end up building a music streaming service, or a forum, the concepts are totally universal. No matter what kind of application we choose to build, we are always going to need a way to mathematically rank what is seen first (Priority Queues), a way to navigate back and forth through media or pages seamlessly (Doubly Linked Lists), and a way to handle background notifications and undo features (Stacks and Queues). So, the architectural foundation we built in this lab is going to carry over perfectly no matter what we end up coding.

thanks.